CINEMAX

Movie Information Portal – Project Explanation Guide

1. Project Overview

Cinemax is a frontend Movie Information Portal developed according to the provided Software Requirements Specification for Web Technology–I. The portal lets users browse, search, filter, sort and view movie information, maintain a favorites list, submit ratings and reviews, and send feedback. The project uses HTML5, CSS3, JavaScript and DOM concepts, with LocalStorage used for selected client-side features.

2. Pages in the Website

File	Purpose
index.html	Home page with banner, featured movies, popular movies and genres.
movies.html	Dynamic movie listing with search, filters and sorting.
movie-details.html	Shows complete information for a selected movie and trailer.
favorites.html	Displays movies saved by the user.
reviews.html	Review/rating submission form and dynamic review display.
contact.html	Contact and feedback form with validation.

3. Main Modules

• Home – banner, featured/popular movies, genres and navigation.

• Movie Listing – cards showing poster, title, genre, year, rating and description.

• Movie Details – title, poster, description, genre, year, language, duration, director, cast, rating and trailer.

• Search & Filter – title/keyword search plus genre, language, year and rating filters.

• Sort – name A–Z/Z–A, rating high/low and year newest/oldest.

• Favorites / Watchlist – add and remove movies; saved in LocalStorage.

• Reviews & Ratings – validated form and dynamic review display.

• Contact / Feedback – validated name, email, subject and message form.

• Advanced feature – dark/light mode and LocalStorage.

4. How the Website Works

Step 1: movies.js contains an array of 15 movie objects. Each object stores the movie ID, title, poster, genre, release year, language, duration, director, cast, description, rating and trailer.

Step 2: JavaScript functions create movie cards from those objects. This avoids writing every movie card manually in HTML.

Step 3: The Movies page reads search and filter controls, filters the array and displays the matching cards through the DOM.

Step 4: Clicking View Details passes the movie ID in the URL. movie-details.html then finds that object and creates the details section dynamically.

Step 5: Favorites and reviews are stored in browser LocalStorage, so they remain available after refreshing the page on the same browser.

5. Technologies Used

HTML5: page structure and forms. CSS3: layout, colors, responsive design, transitions and light/dark theme. JavaScript: arrays, objects, functions, conditions, loops, events and validation. DOM: dynamic cards, details, filters and reviews. LocalStorage: favorites, reviews and theme preference. The SRS also specifies jQuery and Bootstrap for later project phases; the current website is structured so those concepts can be integrated in the next phase.

6. Suggested Professor Demonstration

• Start on Home and explain the navigation and banner.

• Open Movies and demonstrate search, genre/language/year/rating filters and sorting.

• Open a movie and explain how its ID is used to display dynamic details.

• Add a movie to Favorites, refresh the page, and show that LocalStorage keeps it.

• Open Reviews, submit a valid review and show the review appearing dynamically.

• Open Contact and demonstrate required-field and email validation.

• Use the theme button to demonstrate the advanced dark/light mode feature.

• Resize the browser window to demonstrate responsive design.

7. Short Viva Answers

Why JavaScript arrays? To store the movie collection as structured objects and generate cards dynamically.

What is DOM used for? It allows JavaScript to select, create and update HTML elements without manually rewriting the page.

Why LocalStorage? It provides simple client-side persistence for favorites, reviews and theme settings without a backend database.

How does search work? The entered text is converted to lowercase and checked against movie title, genre and description.

How is validation done? HTML required/minlength/type attributes provide basic validation and JavaScript checks email format and submission logic.

Why responsive CSS? The SRS requires the portal to support desktop, laptop, tablet and mobile screen sizes.

8. Project Structure

```
Cinemax_Movie_Portal/
■■■ index.html
■■■ movies.html
■■■ movie-details.html
■■■ favorites.html
■■■ reviews.html
■■■ contact.html
■■■ css/style.css
■■■ js/movies.js
■■■ js/app.js
■■■ js/home.js
■■■ js/movies-page.js
■■■ js/details.js
```

■■■ js/favorites.js
■■■ js/reviews.js
■■■ js/contact.js